IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Compte rendu TP2

TP2 Linux / Premières Nodes

ALIANA FOUDA Jules

Amer Zaoula

Maximiliano Florez

Myrella Alves

Oussama Makni

Sommaire

1. Création d'un workspace catkin **3**
2. Création d'un package catkin **4**
3. Création de nodes **5**
Démarrer un Master ROS **6**
Exécuter un script Python dans ROS **6**
Compilation et exécution d'un Subscriber en C++ **7**
Transfert et compilation du fichier 7
Exécution et tests **9**
4. Création du package turtledriver **10**
Implémentation d'un contrôleur pour la tortue (Turtlesim) **10**
Transfert et compilation du code 10
Explication de la partie implémentée et ajoutée au code **17**
1. Définition et gestion des variables **17**
2. Mécanisme de réception des positions **17**
3. Implémentation du contrôleur de la tortue **17**
4. Gestion des services ROS **18**
5. Boucle principale et exécution **18**
Tests finaux 19
6.Conclusion **20**

Introduction

L'objectif de ce TP est de se familiariser avec l'environnement de travail du ROS en créant un premier couple de **Node ROS** qui s'exécutera sur le raspberry, parallèlement au **roscore. Un topic dans ROS** est un canal de communication utilisé pour échanger des messages entre un émetteur et un receveur **.** Le couple de **Node** en question sera composé d'un **publisher** écrit en Python et d'un **subcriber** écrit en C++ . Par la suite , nous allons créer un autre **Node** pour pouvoir piloter et exécuter les commandes de la tortue vu dans le TP précédent.

1. Création d'un workspace catkin

La commande **"catkin_make"** permet d'initialiser le workspace en y créant les dossiers et fichiers nécessaires.

```
pi@raspberrypi:~ $ mkdir =p ~/my_ws/src
mkdir: impossible de créer le répertoire « /home/pi/my_ws/src »: Le fichier existe
pi@raspberrypi:~ $ cd ~/my_ws/
pi@raspberrypi:~/my_ws $ catkin_make
Base path: /home/pi/my_ws
Source space: /home/pi/my_ws/src
Build space: /home/pi/my_ws/build
Devel space: /home/pi/my_ws/devel
Install space: /home/pi/my_ws/install
####
#### Running command: "make cmake_check_build_system" in "/home/pi/my_ws/build"
####
####
#### Running command: "make -j4 -l4" in "/home/pi/my_ws/build"
####
```

En indiquant la référence du fichier **"devel/setup.bash"** , qui a été lors de l'exécution de la commande catkin_make, on rajoute les chemins d'accès aux packages que l'on crée. En exécutant la commande **"echo $ROS_PACKAGE_PATH" ,** on peut afficher ses chemins d'accès .

```
pi@raspberrypi:~/my_ws $ source devel/setup.bash
pi@raspberrypi:~/my_ws $ echo $ROS_PACKAGE_PATH
/home/pi/my_ws/src:/opt/ros/melodic/share
```

2. Création d'un package catkin

- **Que fait la commande catkin_create_pkg ?**

  La commande `catkin_create_pkg` permet de créer un nouveau package ROS basé sur `catkin`. Elle génère la structure de base du package, y compris un fichier `CMakeLists.txt` et un fichier `package.xml`.

- **Quel est le nom de package que vous venez de créer ?**

  Le package créé s'appelle **`tp2test`**, comme spécifié dans la commande :

  catkin_create_pkg tp2test std_msgs rospy roscpp

- **Quels sont ses dépendances ?**

  Le package dépend des bibliothèques suivantes :

  - **`std_msgs` : Contient des messages standard de ROS (chaînes, entiers, flottants, etc.).**
  - **`rospy` : Bibliothèque Python pour ROS.**
  - **`roscpp` : Bibliothèque C++ pour ROS.**

3. Création de nodes

Nous avons commencé à écrire le code via un éditeur de texte (Visualcode)

```
PublisherNode.py 2
C: > Users > ALIANA FOUDA > Desktop > Cours > IMT-Cours > ROS > PublisherNode.
1   #!/usr/bin/ env python
2   import rospy
3   from std_msgs.msg import String
4   def talker():
5       pub = rospy.Publisher('chatter', String, queue_size=10)
6       rospy.init_node('talker',anonymous=True)
7       rate = rospy.rate(10)
8       while not rospy.is_shutdown():
9           hello_str = "hello worl %s" % rospy.get_time()
10          rospy.loginfo(hello_str)
11          pub.publish(hello_str)
12          rate.sleep()
13
14      if __name__ == '__main__':
15          try:
16              talker()
17          except rospy.ROSInterruptException:
18              pass
```

Figure 1 : Code Publisher Node

```
C: > Users > ALIANA FOUDA > Desktop > Cours > IMT-Cours > ROS > SubscriberNode.cpp > ...
1   #include "ros/ros.h"
2   #include "std_msgs/String.h"
3
4   void chatterCallback(const std_msgs::String:: ConstPtr&msg)
5   {
6       ROS_INFO("I heard : [%s]", msg->data.c_str())
7   }
8   int main (int argc, char **argv)
9   {
10      ros :: init (argc, argv, "listener");
11      ros :: NodeHandle n;
12      ros :: Subscriber sub = n.subscribe("chatter",1000,chatterCallback);
13      ros :: spin ();
14      return 0;
15      }
```

Figure 2 : Subscriber Node

Par la suite , nous avons importer le code dans la carte raspberry à l'aide de la commande ci dessous :

```
PS C:\Users\Lenovo> scp .\ControlTortuga.cpp pi@raspberrypi.local:/home/pi/my_ws/src/tp2test/src
pi@raspberrypi.local's password:
ControlTortuga.cpp                                            100% 4594     4.5KB/s   00:00
PS C:\Users\Lenovo>
```

Figure 3 : Importation du code

Après avoir transféré les fichiers de code vers notre Raspberry (`PublisherNode.py`, `SubscriberNode.cpp` et `Controltortuga.cpp`), nous nous plaçons dans le dossier `my_ws` et exécutons la commande suivante pour compiler notre projet avec Catkin :

```
cd ~/my_ws && catkin_make
```

Une fois la compilation terminée sans erreur, nous devons exécuter la commande suivante :

```
cd ~/my_ws && source ./devel/setup.bash
```

Cette commande nous permet de revenir dans le dossier `my_ws` et d'exécuter un *sourcing* afin d'appliquer les configurations de l'environnement ROS.

Démarrer un Master ROS

Avant d'exécuter toute fonction dans ROS, il est indispensable de démarrer un **Master ROS**, qui gère la communication entre les nœuds. Pour cela, ouvrez un terminal et exécutez la commande suivante :

```
roscore
```

Le *Master* doit rester actif en arrière-plan tant que vous utilisez ROS.

Exécuter un script Python dans ROS

Ensuite, nous pouvons exécuter notre code dans le package `tp2test`. Il est important de noter que pour les scripts Python, il n'est pas nécessaire d'avoir un fichier `CMakeLists.txt` pour les exécuter. Nous pouvons directement lancer le script avec la commande suivante :

```
rosrun tp2test PublisherNode.py
```

- Commentez abondamment le script Python en vous appuyant sur la documentation de `rospy` et `std_msgs`.

Le script Python semble être un Publisher ROS qui publie des messages. Voici une explication ligne par ligne :

Publicador.py × | suscriptor.cpp 3 | activestate.yaml | ControlTortuga.cpp

Publicador.py

```python
import rospy  # Import de la librairie rospy pour les nœuds ROS en Python
from std_msgs.msg import String  # Import du type de message "String" du package std_msgs

# Fonction principale
def talker():
    pub = rospy.Publisher('chatter', String, queue_size=10)  # Création du Publisher sur le topic "chatter"
    rospy.init_node('talker', anonymous=True)  # Initialisation du nœud avec le nom 'talker'
    rate = rospy.Rate(10)  # Fréquence d'envoi des messages (10 Hz)

    while not rospy.is_shutdown():
        hello_str = "Hello ROS World %s" % rospy.get_time()  # Message à publier
        rospy.loginfo(hello_str)  # Affiche le message dans le terminal
        pub.publish(hello_str)  # Publie le message sur le topic
        rate.sleep()  # Pause pour respecter la fréquence de publication

if __name__ == '__main__':
    try:
        talker()  # Appelle la fonction principale
    except rospy.ROSInterruptException:
        pass  # Gestion des erreurs en cas d'interruption
```

Figure 4 : code comenté

Compilation et exécution d'un Subscriber en C++

Maintenant que nous avons un **Publisher** qui envoie des messages sur le topic `/chatter`, nous devons implémenter un **Subscriber** pour écouter ces messages. Celui-ci est écrit en **C++** sous le fichier `SubscriberNode.cpp`.

Transfert et compilation du fichier

Comme illustré dans l'image, avant d'exécuter ce programme, nous devons transférer le fichier source sur notre Raspberry Pi :

scp SubscriberNode.cpp pi@raspberrypi.local:/home/pi/my_ws/src/tp2test/src

Puis, nous devons nous connecter à la Raspberry Pi et accéder au répertoire contenant le package :

cd /home/pi/my_ws/src/tp2test

Édition du fichier `CMakeLists.txt` à l'aide de la commande: `nano CMakeLists.txt` pour s'assurer que notre **SubscriberNode** sera bien compilé :

add_executable(SubscriberNode src/SubscriberNode.cpp)

target_link_libraries(SubscriberNode ${catkin_LIBRARIES})

add_dependencies(SubscriberNode ${catkin_EXPORTED_TARGETS})

Enfin, nous compilons avec :

cd ~/my_ws && catkin_make

- Décrivez les différentes lignes du fichier `CMakeLists.txt`.

  Dans `CMakeLists.txt`, nous ajoutons ces lignes :

  - add_executable(SubscriberNode src/SubscriberNode.cpp)

    Crée un exécutable appelé `SubscriberNode` à partir du fichier source `SubscriberNode.cpp`.

  - target_link_libraries(SubscriberNode ${catkin_LIBRARIES})

    Associe l'exécutable `SubscriberNode` aux bibliothèques Catkin.

  - add_dependencies(SubscriberNode ${catkin_EXPORTED_TARGETS})

    Ajouter des dépendances pour garantir une bonne compilation.

- Commentez abondamment le code source C++ en utilisant la documentation de `roscpp` et `std_msgs`.

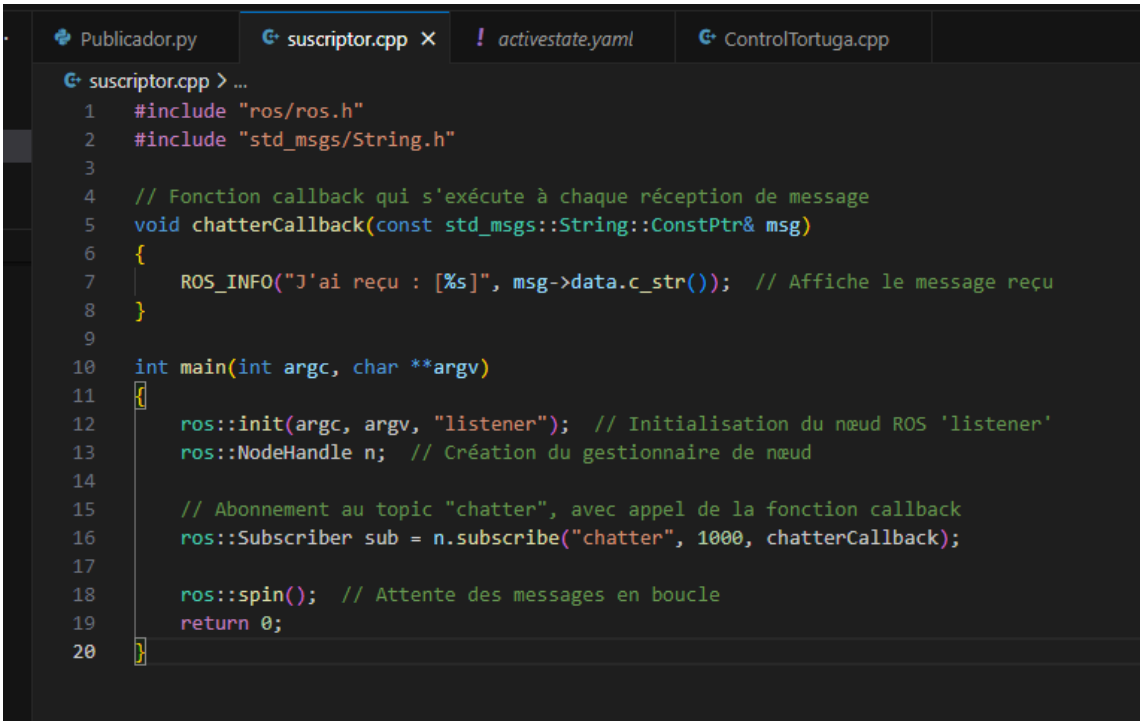

Figure 5 : code commenté

Exécution et tests

Une fois la compilation terminée, nous devons exécuter nos scripts pour tester leur interaction

1. Ouvrir un terminal et exécuter le Publisher :

   rosrun tp2test PublisherNode.py

2. Ouvrir un autre terminal et exécuter le Subscriber :

   rosrun tp2test SubscriberNode

Si tout fonctionne correctement, le terminal du Subscriber affichera les messages envoyés par le Publisher comme cela se montre sur la figure 5.

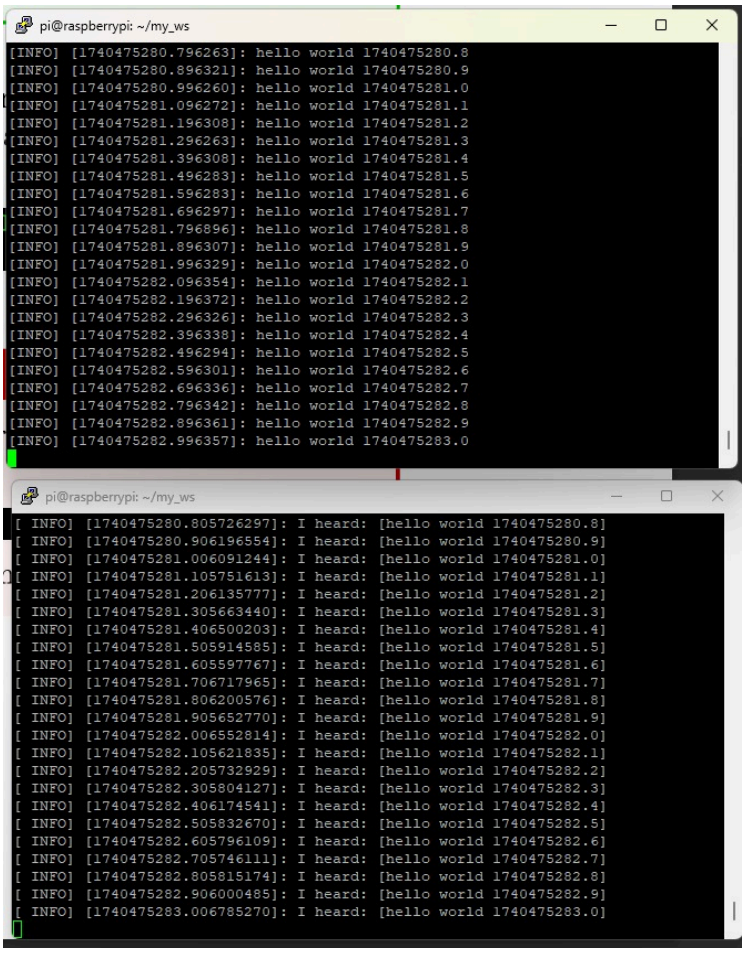

Figure 6 : Visualisation

4. Création du package turtledriver

Implémentation d'un contrôleur pour la tortue (Turtlesim)

Enfin, nous devons compléter le programme `Controltortuga.cpp` qui permet de déplacer la tortue vers un objectif.

Transfert et compilation du code

Comme indiqué dans l'image, nous devons éditer `CMakeLists.txt` pour inclure le nouveau fichier :

add_executable(Controltortuga src/Controltortuga.cpp)

target_link_libraries(Controltortuga ${catkin_LIBRARIES})

add_dependencies(Controltortuga ${catkin_EXPORTED_TARGETS})

Puis compiler avec :

cd ~/my_ws && catkin_make

- Complétez le code pour réaliser un contrôleur permettant à la tortue d'atteindre sa cible.

```
#include <ros/ros.h>

#include <turtlesim/Pose.h>

#include <turtlesim/SetPen.h>

#include <geometry_msgs/Twist.h>

#include <std_srvs/Empty.h>

#include <math.h>       // définit atan2, sqrt et pow

#include <stdlib.h>     // définit rand

#include <iostream>     // définit cout
```

```
// Définir pi comme une constante flottante

const float pi = 3.14159265;


// Définir la position actuelle de la tortue et la position cible comme
variables globales

turtlesim::Pose currentpose;

turtlesim::Pose goalpose;


const float distance_tolerance = 0.1;

const float kp_linear = 1.5;

const float kp_angular = 4.0;


// Fonction pour construire et publier des messages Twist

void setturtlevelocities(ros::Publisher twist_pub, float linear, float
angular)

{

    std::cout << "Définir la vitesse de la tortue à x = " << linear

              << " rotz = " << angular << '\n';


    geometry_msgs::Twist twist;

    twist.linear.x = linear;

    twist.angular.z = angular;


    twist_pub.publish(twist);

}
```

```
// Callback pour mettre à jour la position actuelle de la tortue

void getturtleposecallback(const turtlesim::Pose& pose)

{

    std::cout << "Position actuelle de la tortue :\n" << pose << '\n';

    currentpose = pose;

}



// Callback pour mettre à jour la position cible de la tortue

void getturtlegoalcallback(const turtlesim::Pose& goal)

{

    std::cout << "Objectif de la tortue :\n" << goal << '\n';

    goalpose = goal;

}



// Contrôleur de la tortue

 ////////////////////////////////////

 ////////////////////////////////////



void turtlecontroler(ros::Publisher twist_pub)

{

    float turtle_x = currentpose.x;

    float turtle_y = currentpose.y;

    float turtle_theta = currentpose.theta;

    float goal_x = goalpose.x;

    float goal_y = goalpose.y;
```

```
      float distance = sqrt(pow(goal_x - turtle_x, 2) + pow(goal_y -
turtle_y, 2));

    float desired_angle = atan2(goal_y - turtle_y, goal_x - turtle_x);

    float angle_error = desired_angle - turtle_theta;


    while (angle_error > pi) angle_error -= 2 * pi;

    while (angle_error < -pi) angle_error += 2 * pi;


    float forward_velocity = 0.0;

    float angular_velocity = 0.0;


    float max_linear_speed = 2.0;

    float max_angular_speed = 2.5;


    if (distance > distance_tolerance) {

                  forward_velocity = std::min(kp_linear * distance,
max_linear_speed);

         angular_velocity = std::min(std::max(kp_angular * angle_error,
-max_angular_speed), max_angular_speed);


        if (fabs(angle_error) < 0.05) { // Éviter les oscillations

            angular_velocity = 0.0;

        }

    } else {

        forward_velocity = 0.0;
```

```
        angular_velocity = 0.0;
    }

    setturtlevelocities(twist_pub, forward_velocity, angular_velocity);
}

////////////////////////////////
////////////////////////////////

int main(int argc, char** argv)
{
    // Définir une position initiale en dehors de la zone visible
    goalpose.x = -5;
    goalpose.y = -5;

    // Initialiser ROS
    ros::init(argc, argv, "auto_draw");
    ros::NodeHandle nh;

    // Définir la fréquence de mise à jour : 10 Hz
    ros::Rate r(10);

    // Souscription au topic de la position de la tortue
```

```
        ros::Subscriber  pose_sub  =  nh.subscribe("turtle1/pose",  1,
getturtleposecallback);


    // Souscription au topic de l'objectif de la tortue

      ros::Subscriber  goal_sub  =  nh.subscribe("turtledriver/goal",  1,
getturtlegoalcallback);


    // Création d'un Publisher sur le topic de commande de vitesse

                                ros::Publisher        twist_pub        =
nh.advertise<geometry_msgs::Twist>("turtle1/cmd_vel", 1);


    // Client pour réinitialiser la simulation

                            ros::ServiceClient       resetclient       =
nh.serviceClient<std_srvs::Empty>("reset");

    std_srvs::Empty resetservice;

    resetclient.waitForExistence();

    resetclient.call(resetservice);


    // Client pour modifier la couleur du stylo

                             ros::ServiceClient       penclient       =
nh.serviceClient<turtlesim::SetPen>("turtle1/set_pen");

    penclient.waitForExistence();


    // Initialiser la couleur du stylo

    turtlesim::SetPen pen;

    pen.request.r = 255;

    pen.request.g = 255;
```

```
    pen.request.b = 255;

    pen.request.width = 5;

    pen.request.off = 0;


    while (ros::ok())

    {

        // Modifier la couleur du stylo aléatoirement

        pen.request.r = rand() % 256;

        pen.request.g = rand() % 256;

        pen.request.b = rand() % 256;

        penclient.call(pen);


        // Appeler le contrôleur de la tortue

        turtlecontroler(twist_pub);


        // Envoyer et recevoir des messages ROS

        ros::spinOnce();


        // Attendre selon la fréquence définie

        r.sleep();

    }


    return 0;

}
```

Explication de la partie implémentée et ajoutée au code

Dans cette implémentation, nous avons intégré un **contrôleur de la tortue** qui lui permet d'atteindre un objectif donné en ajustant sa vitesse linéaire et angulaire de manière autonome. Voici une explication détaillée des fonctionnalités ajoutées :

1. Définition et gestion des variables

Nous avons ajouté des **variables globales** pour stocker la position actuelle et l'objectif de la tortue :

- `currentpose` : Représente la position actuelle de la tortue.
- `goalpose` : Contient les coordonnées de l'objectif que la tortue doit atteindre.
- `distance_tolerance` : Définit une **marge d'erreur** permettant de considérer que la tortue est arrivée à destination lorsqu'elle est suffisamment proche.
- `kp_linear` et `kp_angular` : Sont des **coefficients de contrôle proportionnel** qui ajustent la vitesse de la tortue en fonction de la distance et de l'orientation par rapport à l'objectif.

2. Mécanisme de réception des positions

Deux **callbacks** ont été ajoutés pour **mettre à jour** la position actuelle de la tortue et celle de l'objectif :

- `getturtleposecallback()` : Met à jour la variable `currentpose` en fonction des messages reçus depuis le topic `turtle1/pose`.
- `getturtlegoalcallback()` : Met à jour la variable `goalpose` lorsqu'une nouvelle position cible est envoyée sur le topic `turtledriver/goal`.

Ces callbacks permettent de toujours connaître **où se trouve la tortue** et **où elle doit aller**, facilitant ainsi le contrôle du mouvement.

3. Implémentation du contrôleur de la tortue

La fonction **`turtlecontroler()`** joue un rôle central : elle ajuste **dynamiquement** la vitesse et l'orientation de la tortue pour qu'elle se dirige efficacement vers son objectif.

1. **Calcul de la distance et de l'orientation**
    - La distance entre la tortue et l'objectif est calculée avec la formule de la distance euclidienne.
    - L'angle que doit suivre la tortue est calculé avec `atan2`, qui donne l'orientation idéale vers la cible.

2. **Correction de l'angle d'orientation**
    - Si l'angle est **supérieur à π ou inférieur à -π**, on le **ramène dans l'intervalle** [−π,π][-π, π][−π,π] pour éviter des mouvements brusques.

3. **Définition des vitesses linéaires et angulaires**
    - Si la tortue **est loin** de l'objectif, elle avance avec une vitesse proportionnelle à la distance restante.
    - Si l'orientation est incorrecte, elle **ajuste son angle** avant de continuer son déplacement.
    - On impose une **vitesse maximale** pour éviter que la tortue ne bouge trop vite ou trop lentement.
    - Pour éviter que la tortue **oscille** autour de l'angle cible, on désactive la vitesse angulaire si l'erreur d'orientation est très petite (`< 0.05 rad`).

4. **Publication des commandes de mouvement**
    - Les vitesses calculées sont envoyées sous forme de **messages Twist**, permettant à la tortue de **se déplacer progressivement vers la cible**.

4. Gestion des services ROS

Deux services ont été intégrés pour améliorer l'expérience utilisateur :

- **Réinitialisation de la simulation**

  Avant de démarrer, on réinitialise la simulation avec le service `reset` pour s'assurer que la tortue commence toujours **dans une position propre**.

- **Modification aléatoire de la couleur du stylo**

  Un service `turtle1/set_pen` est utilisé pour modifier la couleur du trait tracé par la tortue **de manière aléatoire** à chaque itération.

5. Boucle principale et exécution

Une fois ROS lancé, la boucle **`while (ros::ok())`** assure un contrôle continu :

1. **Mise à jour de la couleur du stylo.**
2. **Appel du contrôleur pour ajuster les mouvements de la tortue.**

3. **Envoi et réception des messages de ROS.**
4. **Pause pour maintenir la fréquence de mise à jour à 10 Hz.**

Tests finaux

Pour tester ce programme, nous devons :

1. **Lancer la simulation Turtlesim** :

   rosrun turtlesim turtlesim_node

2. **Exécuter le contrôleur** :

   rosrun tp2test ControlTortuga

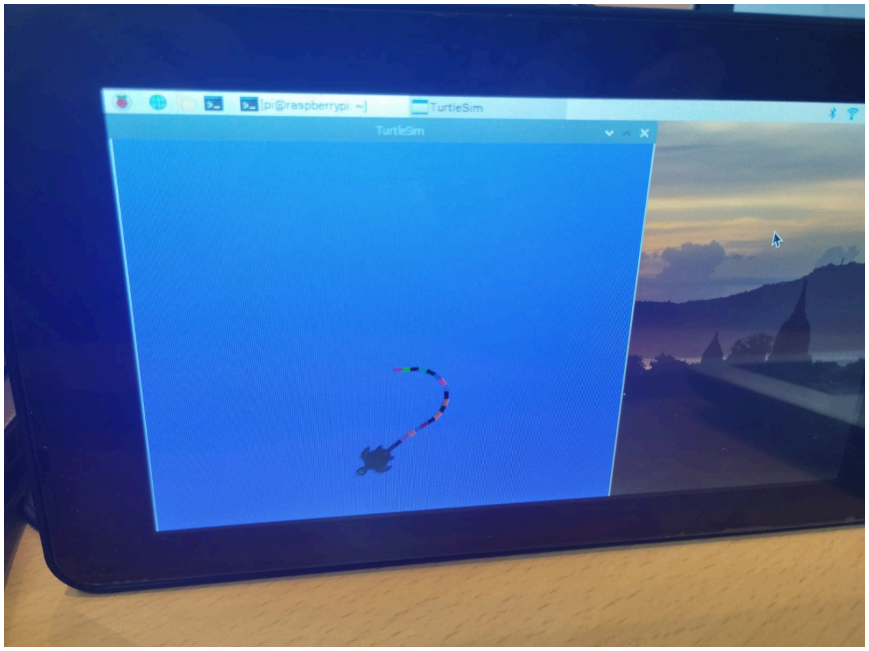

Figure 7 : Simulation du tortue

Conclusion

Dans ce TP, nous avons exploré l'environnement ROS en créant des **nœuds** et en **important des fichiers Python et C++** sur une carte Raspberry Pi. Nous avons utilisé le **protocole SSH** pour communiquer à distance avec celle-ci et exécuter nos scripts directement sur la Raspberry Pi.

Nous avons également intégré un **contrôleur** permettant de piloter la tortue dans le simulateur *turtlesim*. Cette communication entre les différents nœuds nous a permis de tester l'interaction avec ROS en temps réel.

De plus, nous avons approfondi la **gestion des topics** en mettant en place un **Publisher** et un **Subscriber**, facilitant l'échange de messages entre les nœuds ROS. Nous avons aussi manipulé le fichier `CMakeLists.txt` et utilisé des commandes essentielles comme `catkin_make` pour configurer et compiler nos packages ROS.

Enfin, nous avons implémenté un **système de contrôle de la tortue**, permettant d'envoyer des commandes de mouvement via un nœud ROS en utilisant des messages de type `geometry_msgs/Twist` pour gérer sa vitesse linéaire et angulaire. Cette manipulation nous a permis de **tester l'envoi de commandes en temps réel**, d'observer la réponse du système et de **mieux comprendre la communication inter-nœuds dans ROS**.